GOVERNANÇA E COMPLIANCE AMBIENTAL

EcoCompliance API

Pipeline de CI/CD, containerização e deploy em staging e produção.

FIAP — Tecnologia em Análise e Desenvolvimento de Sistemas. Disciplina de DevOps, fase “Navegando pelo mundo DevOps”. A aplicação veio da disciplina de Java Advanced (microsserviços com Spring).

Integrantes

Nome	RM
Évelyn Santos Rodrigues	565919
Fagner Nunes Lopes	564220
Lucas Fernando da Silva	563626
Raphael Salomão Chamma	564765
Ricardo Negrão da Veiga	564106

1. Pipeline

A ferramenta é o GitHub Actions, arquivo **.github/workflows/ci-cd.yml**. Push e pull request em **main** e **develop** disparam o fluxo. Também é possível disparar o deploy manualmente, escolhendo o ambiente.

Etapa	Quando	O que executa
Build	Todo push e pull request	mvn -B -DskipTests package com JDK 21
Testes	Em paralelo com o build	mvn -B test — 165 testes, sem banco
Deploy staging	Push em develop ou disparo manual	Compose com deploy/staging.env, smoke test e teardown no runner
Deploy produção	Push em main ou disparo manual	Compose com deploy/production.env, smoke test e teardown no runner

Pull request só compila e testa. O deploy não roda nessa etapa. No runner o ambiente é efêmero: sobe, valida **GET /ambiente** e o login, e encerra os containers. Na máquina local os dois ambientes permanecem no ar.

2. Docker

O Dockerfile é multi-stage. A primeira etapa, **maven:3.9-eclipse-temurin-21**, gera o jar sem executar testes, porque o job de testes já fez isso. A imagem final, **eclipse-temurin:21-jre-alpine**, copia só o jar, cria o usuário **spring** e expõe a porta 8080.

O Compose sobe a API e o Oracle XE 21 na rede **ecocompliance-net**. O banco persiste no volume **oracle-data**. A API só inicia depois do healthcheck do Oracle. Porta, nome do container e **APP_ENV** vêm de arquivo de ambiente.

Ambiente	API	Oracle no host	Arquivo
Produção	localhost:8080	1521	deploy/production.env
Staging	localhost:8081	1522	deploy/staging.env

Para subir cada ambiente:

```
./scripts/deploy-ambiente.sh deploy/production.env
```

```
./scripts/deploy-ambiente.sh deploy/staging.env
```

3. Evidências

Os dois ambientes responderam ao mesmo tempo em 10/10/2026. O login de admin@ecocompliance.com retornou HTTP 200 em ambos.

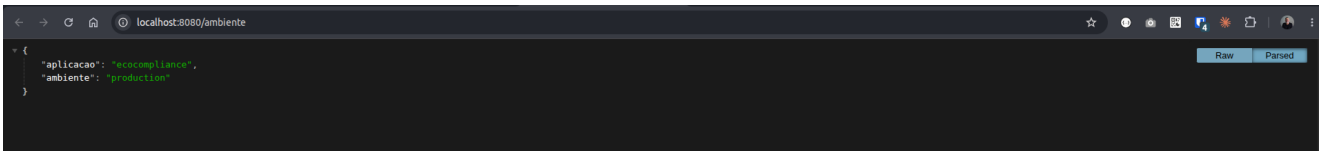


Figura 1. Produção em http://localhost:8080/ambiente.

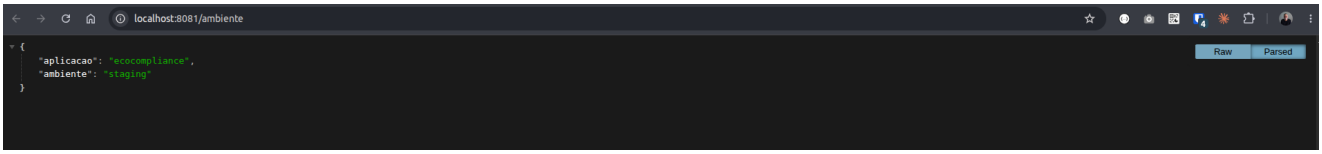


Figura 2. Staging em http://localhost:8081/ambiente.

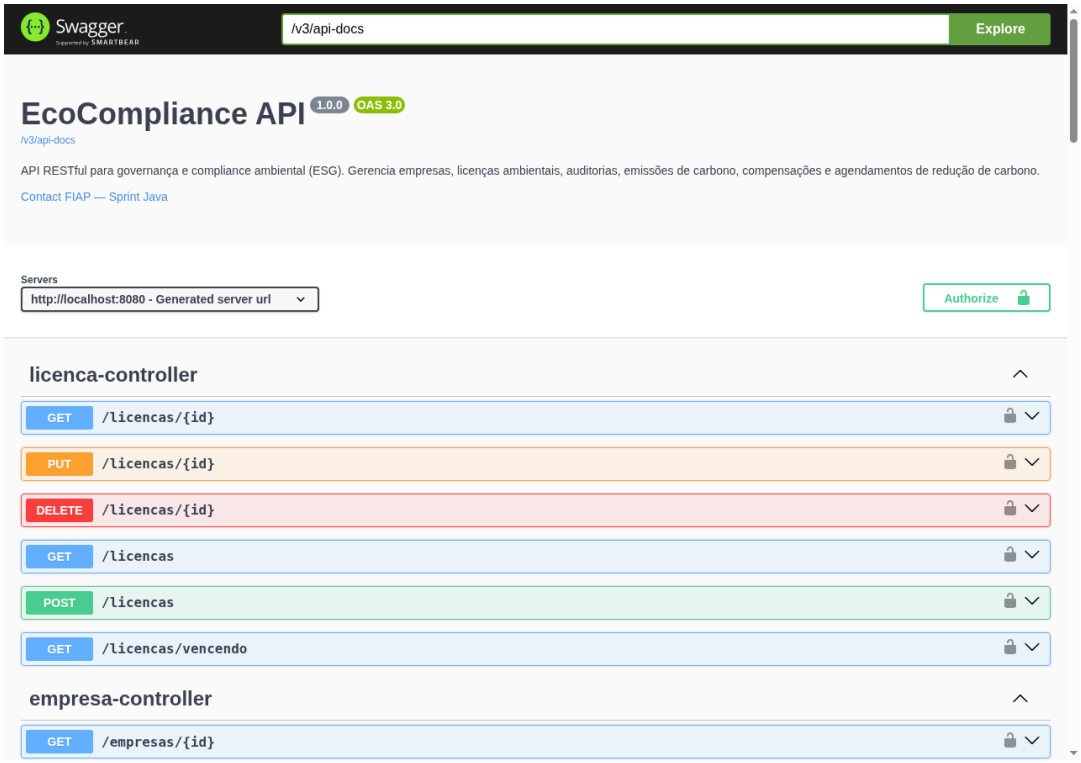


Figura 3. Swagger da produção, servidor em http://localhost:8080.

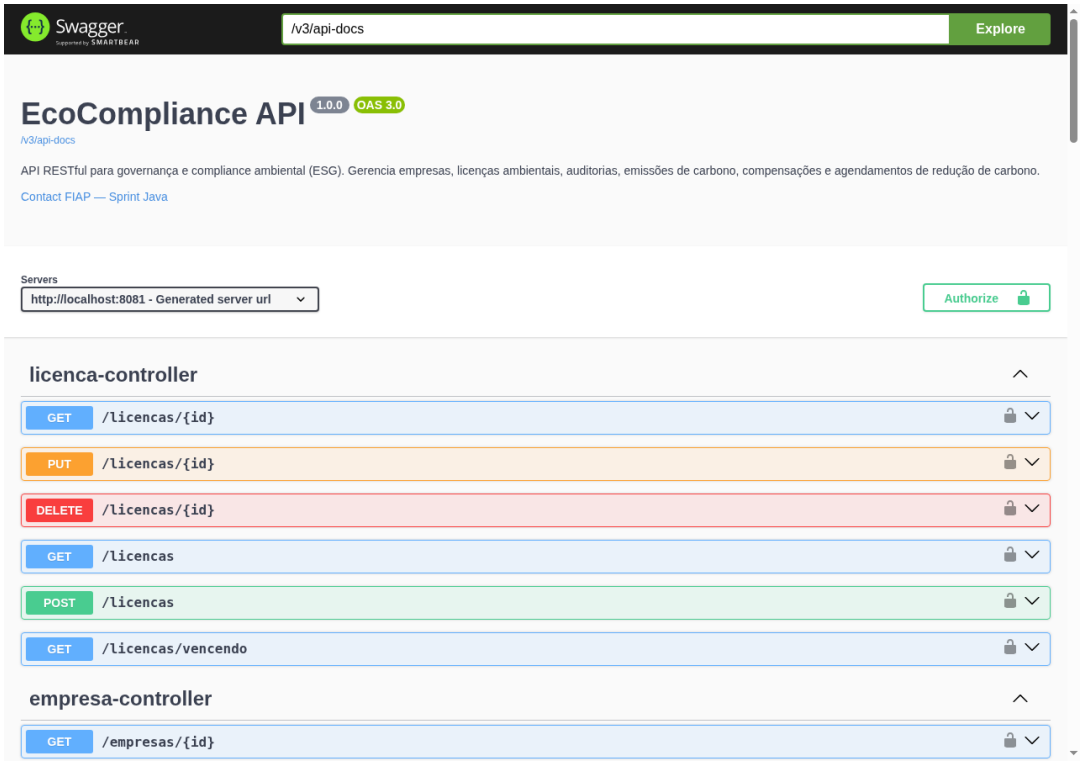


Figura 4. Swagger do staging, servidor em http://localhost:8081.

A tela do GitHub Actions aparece no primeiro push deste workflow para https://github.com/fagnerlopes/ecocompliance-api. As quatro etapas já estão no arquivo do pipeline.

4. Desafios

- A suíte não abre conexão com o Oracle. O job de testes fica rápido e o banco entra só no deploy, quando o Compose precisa provar a aplicação de ponta a ponta.
- Staging e produção usam projeto, volume, rede e porta diferentes. Derrubar um ambiente não apaga o banco do outro.
- A imagem não executa os testes. Quem garante a suíte é o job anterior ao deploy, e a imagem final leva só o JRE e o jar.
- No GitHub Actions o deploy valida e encerra os containers, para o runner não guardar o Oracle. Na máquina de desenvolvimento os dois ambientes continuam disponíveis.

5. Checklist

Item	OK
Projeto compactado em ZIP com estrutura organizada	sim
Dockerfile funcional	sim
docker-compose.yml	sim
Pipeline com build, teste e deploy	sim
README.md com instruções e evidências	sim
Documentação técnica (este PDF)	sim
Deploy em staging e produção	sim